

Sequence Modeling Neural Networks

- Enc2Dec, Attn. Mod., Transformer

S. R. Mahadeva Prasanna

Professor, Dept. of EE, IIT Dharwad

prasanna@iitdh.ac.in

October 14, 2024



Some Good Resources for NNDL

- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024
- Simon J. D. Prince, "Understanding Deep Learning", MIT Press, 2024



Organization

- Encoder-Decoder Architecture
- Attention Modelling
- Transformer network

Seq2Seq Learning using Neural Networks

- Sequence to sequence modeling neural architectures
- RNN & variants one to many and many to one
- Better architectures to take care of many to many
- Encoder-Decoder, attention, & transformer
- Transfer learning
- Language models (LMs), large language models (LLMs)

Different Architectures of RNN

- Input sequence $\mathbf{X}$ length is T_x and output sequence $\mathbf{Y}$ length T_y . T_x need not be equal to T_y
- $T_x \gg 1, T_y \gg 1$ and $T_x = T_y$ or $T_x \neq T_y$, many to many architecture.
 - Machine translation, Matching two seqs. of different length.
- $T_x \gg 1$ and $T_y = 1$, many to one architecture.
 - Binary classification task.
- $T_x = 1$ and $T_y \gg 1$, one to many architecture.
 - Generation of new data using one seed data.
- $T_x = 1$ and $T_y = 1$, one to one architecture.
 - Any conventional PRML task.



Motivation for Sequence 2 Sequence Modelling

- RNN: Modelling information in sequence
- RNN and its variants can handle variable length sequences in input
- Sequence to sequence modeling of variable lengths seems to be difficult for RNN and its variants
- Because in seq2seq modeling, ingesting entire input sequence needs to be done before generating output sequence ‘
- New architectures are needed for doing the same
- Hence the motivation ‘

Encoder-Decoder Architecture

- First architecture designed to handle sequence to sequence modelling
- **Encoder:** Input sequence to context vector
- **Decoder:** Context vector to output sequence
- Encoder analyzes input sequence, understands the whole sequence knowledge and captures essence in context vector
- Decoder analyzes sequence knowledge encoded in context vector and generates output sequence

Inside of Encoder-Decoder

- Encoder will have LSTM / GRU as cells
- Only STC and LTC are derived from the final sequence position as context vector
- No other output in Encoder
- Decoder using context vector having STC and LTC as input and generates output sequence again using LSTM / GRU as cell

Training of Encoder-Decoder Architecture

- Random initialization of encoder-decoder
- Apply input sequence to encoder and generate context vector
- Apply context vector as input to decoder and generate output sequence
- Calculate loss
- Perform backpropagation of error and compute change in weight values
- Update weights
- Repeat process until convergence

Testing of Encoder-Decoder Architecture

- Apply testing input sequence to encoder
- Generate context vector from encoder
- Apply context vector to decoder
- Derive output sequence

Deep Encoder-Decoder Architecture

- Multilayer encoder-decoder ‘

Sequence to Sequence Learning with Neural Networks

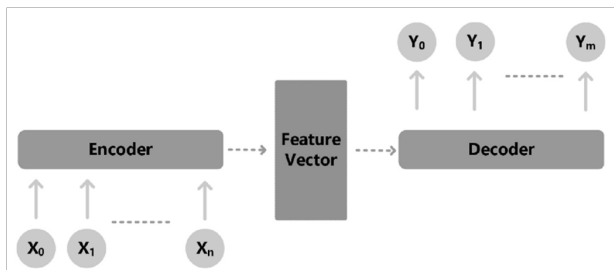
- End-to-end approach to sequence learning that makes minimal assumptions on the sequence structure
- A multilayered LSTM to map input sequence to a vector of a fixed dimensionality
- Another deep LSTM to decode the target sequence from the vector
- LSTM did not have difficulty on long sentences

Ilya Sutskever, et. al, "Sequence to sequence learning with neural networks", NIPS 2014



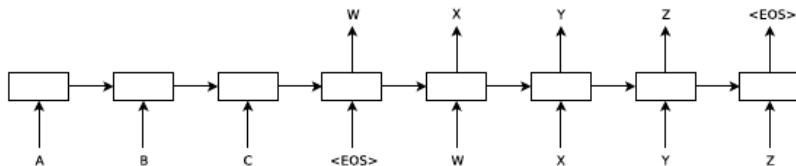
Working Principle of Sequence to Sequence Learning with Neural Networks

- "Use one LSTM to read input sequence, one time step at a time, to obtain large fixed dimensional vector representation, and then use another LSTM to extract output sequence from that vector."



Ilya Sutskever, et. al, "Sequence to sequence learning with neural networks", NIPS 2014

Approach for Sequence 2 Sequence Decoding



Performance of Encoder-Decoder Architecture

Method	test BLEU score (ntst14)
Bahdanau et al. [2]	28.45
Baseline System [29]	33.30
Single forward LSTM, beam size 12	26.17
Single reversed LSTM, beam size 12	30.59
Ensemble of 5 reversed LSTMs, beam size 1	33.00
Ensemble of 2 reversed LSTMs, beam size 12	33.27
Ensemble of 5 reversed LSTMs, beam size 2	34.50
Ensemble of 5 reversed LSTMs, beam size 12	34.81

Table 1: The performance of the LSTM on WMT'14 English to French test set (ntst14). Note that an ensemble of 5 LSTMs with a beam of size 2 is cheaper than of a single LSTM with a beam of size 12.

Performance of Encoder-Decoder Architecture

Method	test BLEU score (ntst14)
Baseline System [29]	33.30
Cho et al. [5]	34.54
Best WMT'14 result [9]	37.0
Rescoring the baseline 1000-best with a single forward LSTM	35.61
Rescoring the baseline 1000-best with a single reversed LSTM	35.85
Rescoring the baseline 1000-best with an ensemble of 5 reversed LSTMs	36.5
Oracle Rescoring of the Baseline 1000-best lists	~45

Table 2: Methods that use neural networks together with an SMT system on the WMT'14 English to French test set (ntst14).

Performance of Encoder-Decoder Architecture

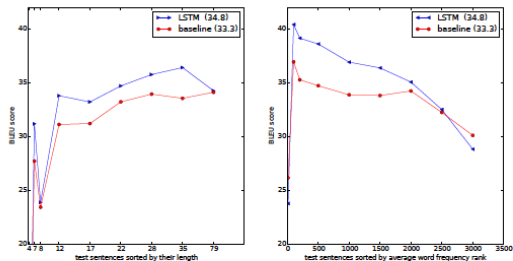


Figure 3: The left plot shows the performance of our system as a function of sentence length, where the x-axis corresponds to the test sentences sorted by their length and is marked by the actual sequence lengths. There is no degradation on sentences with less than 35 words, there is only a minor degradation on the longest sentences. The right plot shows the LSTM's performance on sentences with progressively more rare words, where the x-axis corresponds to the test sentences sorted by their "average word frequency rank".

Limitation of Encoder-Decoder Architecture

- Processing entire input sequence at once and creating context vector is difficult for Encoder-Decoder architecture
- Especially, when sentences are of long length
- Production of any word in the decoder need not depend on entire input sentence
- Only few words out of entire sentence should suffice
- The same is exploited in what is called attention modelling

Limitation of Encoder-Decoder Performance of Encoder-Decoder Architecture

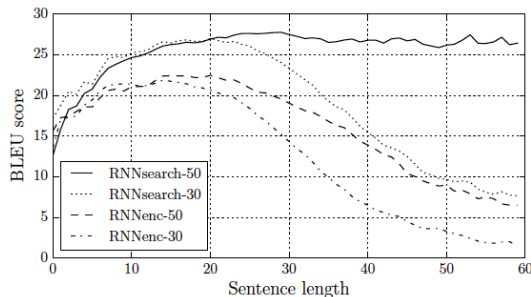


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Attention Modelling

- Learn about neighbourhood that needs attention while generating a particular word in decoder
- From static to dynamic context vector for each decoded entity
- Humans do on the go translation with attention span on few words.

Attention Modelling

- Hidden state values of encoder and decoder are used as inputs
- Output will be dynamic context vector value
- Dynamically select positions relevant for decoding specific output
- For every unit position in decoder, take hidden state values from both encoder and decoder and derive attention value
- Output will be context or attention vector value for that unit

Deriving Attention Vector

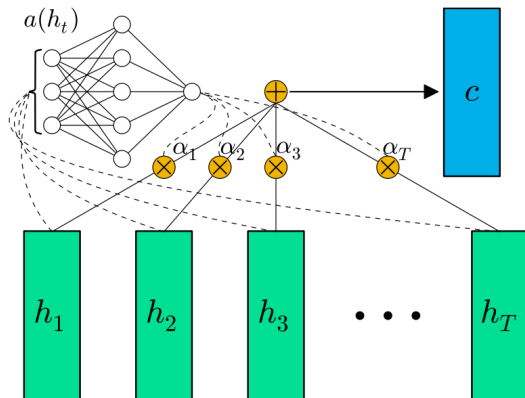


Figure 1: Schematic of our proposed “feed-forward” attention mechanism (cf. (Cho, 2015) Figure 1). Vectors in the hidden state sequence h_t are fed into the learnable function $a(h_t)$ to produce a probability vector α . The vector c is computed as a weighted average of h_t , with weighting given by α .

Attention Modelling - I/O

- **Encoder** : Each HIDDEN state output depends on current input (x) and past hidden state values (h) of encoder LSTM (x, h)
- **Decoder** : Each HIDDEN state output depends on context vector (c) and past output ($y - 1$) of decoder LSTM ($y - 1, c$)
- **Attention Encoder** : Same as in Encoder-Decoder
- **Attention Decoder** : Along with input (h) and past hidden state value (s), attention value is also fed (a), i.e., $f(h, s)$



Deriving Attention Value

- Attention vector will have dimension of decoder length, say N
- $a = [a_1, a_2, \dots, a_N]$
- Each dim of attention vector from specific dim of encoder output
- Let encoder of is of dim M
- $a_1 = \alpha_{11}h_1 + \alpha_{12}h_2 + \dots + \alpha_{1M}h_M$
- $a_2 = \alpha_{21}h_1 + \alpha_{22}h_2 + \dots + \alpha_{2M}h_M$
- $a_N = \alpha_{N1}h_1 + \alpha_{N2}h_2 + \dots + \alpha_{NM}h_M$
- $a_i = \sum_{j=1}^M \alpha_{ij}h_j$



Dependency of α_{ij}

- α_{ij} decides the attention value for a particular decoder step
- α_{ij} depends on both current hidden state of encoder h_j and past hidden state value of decoder s_{i-1}
- $\alpha_{ij} = f(h_j, s_{i-1})$
- How to find f ?
- Derive using a neural network
- Neural network output will be attention vector value
- For every unit position in decoder, take hidden state values from both encoder and decoder and pass it to neural network
- Output of neural network will be context vector value for that unit



Training Neural Network for α_{ij}

- Integrate neural network along with encoder and decoder
- Neural network will be sandwich between encoder and decoder
- Train the same using back propagation through time

Bhadanau Attention for finding α

- Neural network based method described earlier is this mechanism

Luong Attention for finding α

- Non-neural network approach
- Current hidden state value of decoder, where as it was previous hidden state value
- Current value gives more needed information than past
- Dot product of hidden state values of encoder and decoder
- Connect attention value to output of decoder rather than input

Illustrations of Attention Modelling

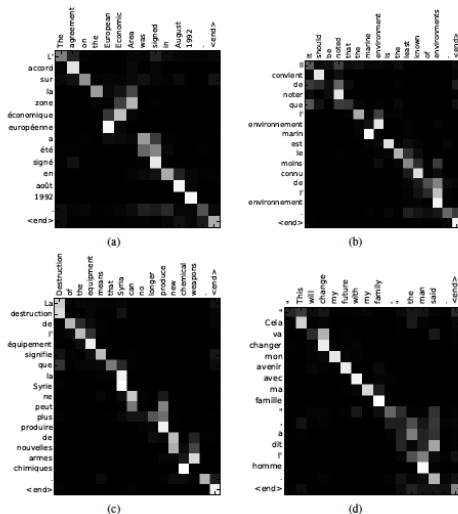


Figure 3: Four sample alignments found by RNNsearch-50. The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j -th source word for the i -th target word (see Eq. (6)), in grayscale (0: black, 1: white). (a) an arbitrary sentence, (b-d) three randomly selected samples among the sentences without any unknown words and of length between 10 and 20 words from the test set.

Performance of Attention Modelling

Model	All	No UNK ^o
RNNencdec-30	13.93	24.19
RNNsearch-30	21.50	31.44
RNNencdec-50	17.82	26.71
RNNsearch-50	26.75	34.16
RNNsearch-50*	28.45	36.15
Moses	33.30	35.63

Table 1: BLEU scores of the trained models computed on the test set. The second and third columns show respectively the scores on all the sentences and, on the sentences without any unknown word in themselves and in the reference translations. Note that RNNsearch-50* was trained much longer until the performance on the development set stopped improving. (o) We disallowed the models to generate [UNK] tokens when only the sentences having no unknown words were evaluated (last column).

Limitation of Attention Modelling

- Sequential mode of training, slow, time consuming, & parallelism is difficult
- Training on large datasets is a challenge
- Pre-training and transfer learning are not possible as in the case of CNNs used in object classification tasks
- Need to train model from scratch for every task (time, cost)
- Need a new neural architecture for sequence 2 sequence modeling that overcomes these limitation
- This lead to Transformer architecture, transforms one sequence to another sequence and hence the name

Transformer

- Proposed in landmark paper on "Attention is all you need" in 2017
- Another encoder-decoder architecture
- Only difference is it does not involve LSTM inside thus avoiding sequential training
- Whole architecture is based on attention mechanism termed as self-attention
- Internally uses many neural network concepts to ensure realizing parallelism
- As a result, can train on very large datasets
- Can exploit transfer learning



Transformers

- A new simple network architecture based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.
- Demonstrate superior in quality while being more parallelizable and requiring significantly less time to train.
- Significant saving in computations is achieved by converting sequential operations into matrix / tensor multiplications
- Same is achieved using Self-attention
- Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence.



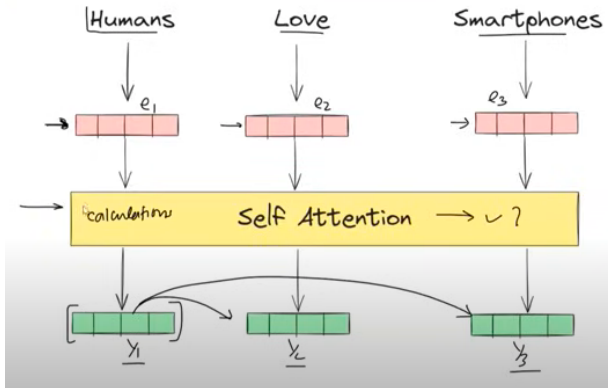
Vectorization of Natural Data

- Vectorization of natural data
- One hot encoding (OHE), bag of words (BoW), Term-Frequency - inverse document frequency (TF-IDF)
- Word embeddings by neural networks - better than OHE, BoW, TF-IDF as it captures semantic meaning
- Different dim will have different meanings
- Two words closer semantically will be similar embedding vectors

Motivation for Self-Attention

- Word embeddings represents average meaning based on input data
- Vectors are static which will lead to improper usage
- Need smart contextual embeddings, i.e., embedding vector dim values change based on context
- Self-attention helps in dealing with this
- Static word embeddings for each word are given as input to self-attention block
- Self-attention block recomputes the dynamic embeddings for each of the words based on the context in the given sequence

Pictorial Self-Attention Process

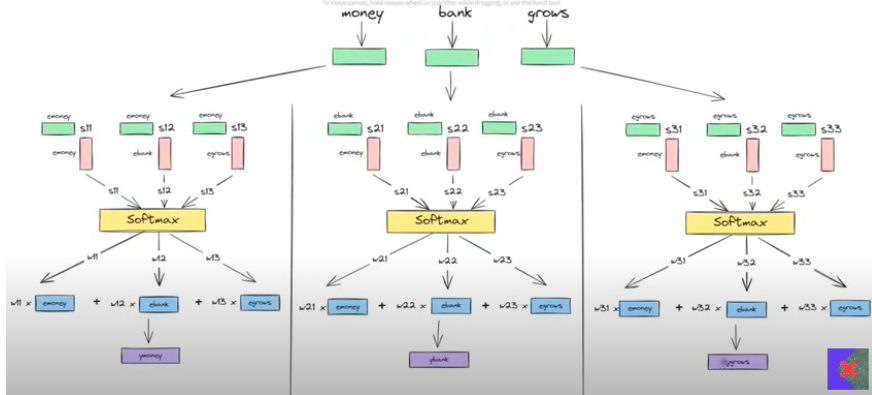


Why it is called Self-Attention?

- Based on input embeddings, self-attention module computes and provides smart contextual embeddings
- New smart contextual embeddings will be a linear combination of input static embeddings
- Attention because, trying to find out which is important to generate a given word
- It is self because, you are deriving attention vector comparing with itself (same sentence)
- Inter-sequence vs intra-sequence attention

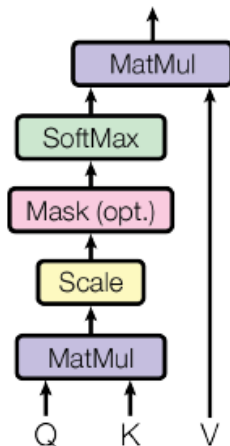
Computation of Contextual Vectors

To move canvas, hold mouse wheel or scroll bar while dragging, or use the hand tool



Scaled Dot Product Attention

Scaled Dot-Product Attention

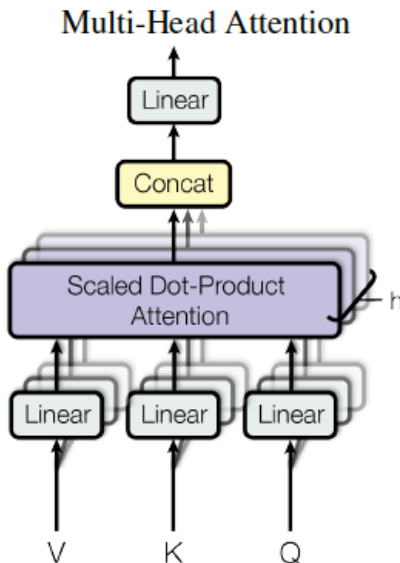


Self-Attention Mechanism

- To bring parallelism, computing attention based representation for each entity in the input example.
- Each word embedding vector can play role of query, key and value
- Query matrix $Q = w^q x$, Key matrix $K = w^k x$, and Value matrix $V = w^v x$, where, Q, K, V are of dimension d_q, d_k, d_v , respectively.
- Weight values are task specific learnt information
- Attention value for each entity, $A_i(q, k, v) = \text{softmax}(\frac{QK^T}{\sqrt{d^k}}) V$.
- Value represents the weightage of given entity to the compared one.
- Key represents type of answer to be retrieved for given query.



Multi-Head Attention



Multi-head Attention Mechanism

- Same word or entity will play different roles (ex., subject, object, verb)
- To account for same, multi-head attention
- Multiple weight matrices each for Q, K, V .
- **Head 1** Query matrix $Q_1 = w_1^q x$, Key matrix $K_1 = w_1^k x$, and Value matrix $V_1 = w_1^v x$,
- **Head 2** Query matrix $Q_2 = w_2^q x$, Key matrix $K_2 = w_2^k x$, and Value matrix $V_2 = w_2^v x$,
- Attention value for each entity,
 $M_i(q, k, v) = \text{concat}(\text{Head}1, \text{Head}2, \dots)$.
- Where $\text{Head}i$, $\text{Head}_i(q, k, v) = \text{softmax}(\frac{QK^T}{d^k})V$.



Transformer - Model Architecture

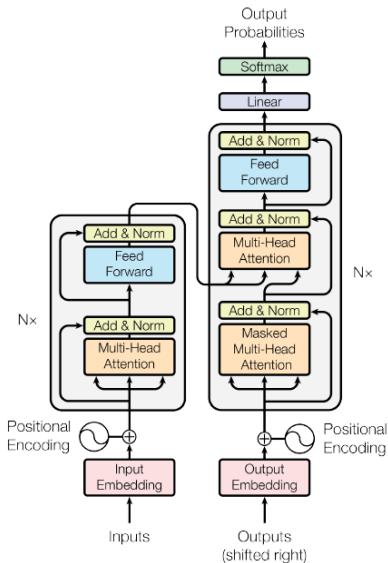


Figure 1: The Transformer - model architecture.

Transformer Network

- Training / Encoding : Multi-head attention network, feedforward network and position encoding
- Testing / Decoding : Multi-head attention network and reuse of key (K) and value (V) matrices from Encoder for better decoding.
- Further the decoder will have softmax layer to decode the words

Transformer Encoder

- Transformer also follows Encoder-Decoder framework
- Follows stacked self-attention and point-wise, fully connected layers for both the encoder and decoder
- **Encoder:** Stack of $N = 6$ identical layers.
- Each layer has two sub-layers.
- The first is a multi-head self-attention mechanism, and the second is a simple, positionwise fully connected feed-forward network.
- A residual connection around each of the two sub-layers, followed by layer normalization.



Transformer Decoder

- The decoder is also composed of a stack of $N = 6$ identical layers.
- In addition to two sub-layers in each encoder layer, decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack.
- Residual connections around each of the sub-layers, followed by layer normalization.
- Self-attention sub-layer in the decoder stack is modified to prevent positions from attending to subsequent positions.
- This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .



Attention Process in Transformer

- Transformer uses multi-head attention in three different ways
- In "encoder-decoder attention" layers, queries come from previous decoder layer, and memory keys and values come from output of encoder.
- Allows every position in decoder to attend over all positions in input sequence.
- This mimics typical encoder-decoder attention mechanisms in sequence-to-sequence models.
- Encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place.
- Each position in encoder can attend to all positions in previous layer of encoder.



Attention Process in Transformer

- Self-attention layers in decoder allow each position in decoder to attend to all positions in decoder up to and including that position.
- Need to prevent leftward information flow in decoder to preserve auto-regressive property.
- This is implemented inside of scaled dot-product attention by masking out all values in input of softmax which correspond to illegal connections.

Transformer Performance

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Challenges in using Transformers

- Large no. of computations, need for GPUs
- Large amount of data (LLMs)
- Overfitting, if data is less as there are large no of parameters
- Huge energy consumption
- Interpretation is difficult as it is more like a block box
- Bias of model towards training data

Thank You